

slot c

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

26

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code: CSA1402

Course Title: COMPILER DESIGN

CO Assessed: CO1 – Imitation (L1)

Assessment: Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING

Weightage: 40% of CIA

CO1 Statement: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Student Name: A. Midhuna

Reg. No.: 192521302

Section / Batch:

Date: 16/7/26

Code of Conduct (192521302)

I, A. Midhuna (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A. Midhuna

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4 / 4	3 / 4	4 / 4	4 / 4	3 / 4	91.25
2	4 / 4	3 / 4	3 / 4	3 / 4	4 / 4	83.75
3	3 / 4	4 / 4	3 / 4	4 / 4	3 / 4	85
4	4 / 4	4 / 4	3 / 4	3 / 4	3 / 4	85
5	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4	90
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	87 88.5
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

N. D.

Analytical problem solving

Compiler phases:-

$$\text{Gin: } p = (a * b) + (b * c) * 2$$

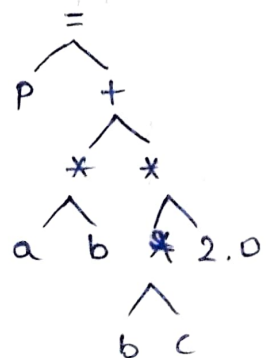
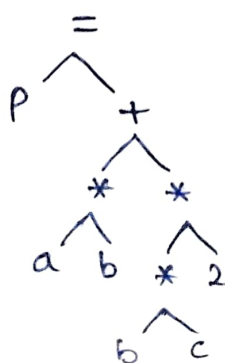
a) Lexical Analysis (Tokens)

Token	Type
p	Identifier
=	Assignment operator
(	Left parenthesis
a	Identifier
*	multiplication
b	Identifier
)	Right parenthesis
+	Addition operator
(	Left parenthesis
b	Identifier
)	Right parenthesis
*	multiplication
2	constant

Total Tokens = 15

c) Semantic Analyzer

b) Syntax tree



d) Intermediate code (Three address code)

temp1 = a * b

temp2 = b * c

temp3 = t2 * 2

temp4 = t1 + t3

P = temp4

e) optimized intermediate code

temp1 = a * b

temp2 = b * c

P = temp1 + temp2 * 2

f) Target code (Eg)

LOAD a

MUL b

STORE R1

LOAD b

MUL c

MUL 2

ADD R1

STORE P

2) Regular Expressions

a) $(\epsilon + aa)^*$

ϵ

aa

aaaa

aaaaaaaa

aaaaaaaaaa

b) $(a + b^*)^* abb$

abb

aabb

babbb

bbabbb

aaabbb

$(a+b)^* aba(a+b)^*$

aba

aaba

abaa

babab

$(a+ab)^*$

a

aa

ab

aab

abab

e) $(aba) + (ab)^*$

aba

abaaba

abaabab

abaab

abaababab

(i) No of tokens

1 - main

11 - b

21 - d

2 - (

12 - =

22 - =

3 -)

13 - "abc"

23 - "xyz"

4 - {

14 - ;

24 - ;

5 - int

15 - int

25 - t

6 - a

16 - c

26 - m

7 - =

17 - =

27 - =

8 - 10

18 - 30

28 - 40.5

9 - ;

19 - ;

29 - ;

10 - char

20 - char

30 - }

Total tokens = 28

(ii) lexical forms:

Errors:

a = lx; assigning address of integer
pointer

1 x ab → Invalid numeric literal

4. Regular Expressions

(i) Even number of 0's

$$1^* (01^* 01^*)^*$$

(ii) Atleast one 1

$$0^* 1 0^*$$

or

$$(0+1)^* 1 (0+1)^*$$

(iii) odd number of 0's and odd number of 1's

$$(00+11)^* (01+10) (00+11)^*$$

(iv) Starts with 1 ends with 0

$$1 (0+1)^* 0$$

(v) No consecutive 1's

$$(0+10)^* 1 ?$$

5) Lexical Analyzer:

a) 1 - int

10 - ;

2 - var123

11 - 16

19 - ;

3 - =

12 - [

20 - }

4 - 12345

13 - var123

5 - ;

14 - ==

6 - float

15 - var456

7 - var456

16 -)

8 - =

17 - {

9 - 123.45

18 - return

b) Tokens by category

Keywords

int
float
if
return

Identifiers

var123
var456
var123
var456

Operators

=
=
==

Separators

;
;
(
)
{
}

Integer literal

12345

Floating

literal
123.45

FSM State Transitions (Approximate)

Token type

Keywords → 20

identifiers → 28

operators → 4

Separators → 7

Integer literal → 5

Floating literal → 7
71

* The FSM starts initial state

* It reads each character every token changes states accordingly

* After recognizing token reaches accepting state return start state next token.

* Summing transition 20 tokens estimated 92 state transitions

Approximately 92 state transitions